Звіт з лабораторної роботи номер 2 з дисципліни “Додаткові розділи с++”
Власюка Назара група к28
варіант 15

Характеристики середовища та початкові дані

AMD Ryzen 5700U: cores 8, Logical processors 16,
RAM 16gb,
Операційна система: Windows 10 Pro
IDE : Compiler: MSVC 2022 (Visual Studio 2022 v17.14, C++23 Standard)
Режими збирання: Release з максимальною оптимізацією (02) та без оптимізації (0d).
Обсяги даних**:** 10 000, 100 000, 1 000 000, 10 000 000 елементів.
Генерація: Використано засоби бібліотеки <random>, дані зчитано з попередньо згенерованих текстових файлів.

Порівняльний аналіз між часом роботи 02 та 0d

Опер.	N	Режим	Без політики	seq	unseq	par	par_unseq	Найкр. K	Час при найкр. K
FastOp	10k	/O2	0,0016	0,0017	0,0017	0,0051	0,0059	1	0,0970
		/Od	0,0176	0,0173	0,0234	0,0365	0,0417	1	0,1071
	100k	/O2	0,0265	0,0167	0,0167	0,0309	0,0361	1	0,1120
		/Od	0,1725	0,1727	0,1727	0,1162	0,1158	2	0,2402
	1M	/O2	1,41	1,38	1,38	1,41	1,42	8	1,59
		/Od	1,99	2,03	2,01	1,64	1,43	8	1,64
	10M	/O2	14,02	13,94	13,93	14,03	14,12	2	14,19
		/Od	29,54	23,11	25,74	14,27	14,03	4	14,43
SlowOp	10k	/O2	9,84	9,84	9,84	0,9217	0,9011	16	1,45
		/Od	10,97	10,97	10,97	0,9994	0,9950	16	1,56
	100k	/O2	98,43	98,47	98,46	8,51	8,43	16	8,97
		/Od	109,7	109,6	109,7	9,36	9,41	32	10,07
	1M	/O2	985,0	984,5	984,7	85,32	84,56	16	85,30
		/Od	1098,1	1098,9	1099,5	92,87	92,27	16	95,29
	10M	/O2	9846,5	9852,4	9851,5	855,5	920,8	16	1042,6
		/Od	10978,7	10971,6	10970,2	928,7	1045,5	16	1144,3

таблиця 2: Залежність часу від K (мс), режим /O2

K	1	2	4	8	16	32	64	128	256	512
SlowOp, 10M	9848	4924	2466	1380	1043	1110	1099	1096	1103	1108
FastOp, 10k	0,097	0,141	0,216	0,486	0,679	1,40	2,81	5,46	11,7	24,5

у першому рядку час падає майже вдвічі при подвоєнні K до 16, а далі виходить на плато (≈1100 мс);

у другому рядку час після K = 16 зростає приблизно вдвічі при кожному подвоєнні K, тобто лінійно.

Відповіді на запитання

Без політики, seq, unseq. Ці три варіанти працюють практично однаково. Для SlowOp різниця менша за 0,2 %. Для FastOp час дуже малий, тому різниця лежить у межах похибки вимірювання. unseq виграшу не дав: sin і cos компілятор не векторизує, а FastOp він, швидше за все, і так векторизує.
par та par_unseq. Для SlowOp вони дають прискорення 10,5–11,9 раза на всіх розмірах. Для

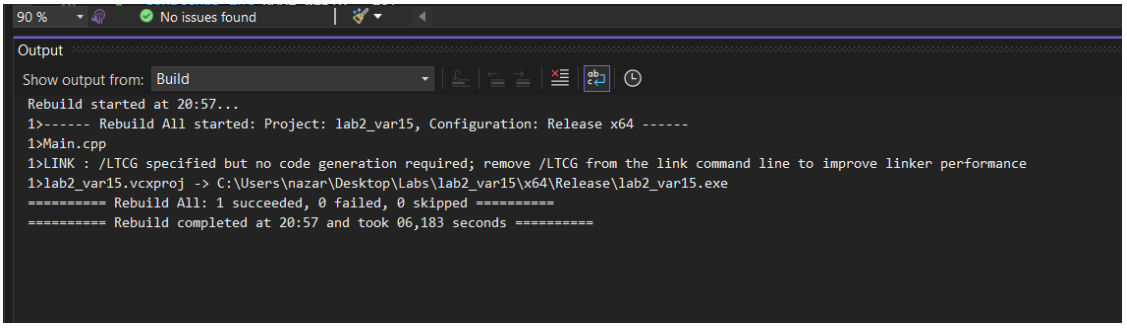
FastOp при /O2 прискорення немає, а на малих даних навіть програш (0,27–0,86 раза), бо запуск потоків коштує дорожче за саму роботу. При /Od на 10M виграш є: 1,6–2,1 раза.

Найкраще K і його зв'язок з потоками процесора. Для SlowOp найкраще K = 16, тобто K/HW = 1,0 (7 випадків із 8, в одному K = 32, але K = 16 лише на 2 % гірше, тож це похибка). До K = 8 прискорення майже лінійне (7,1–7,9 раза), а від 8 до 16 потоків додається ще 20–45 % завдяки SMT. Для FastOp найкраще K мале (1–8), «майже найкраще» 1 або 2.

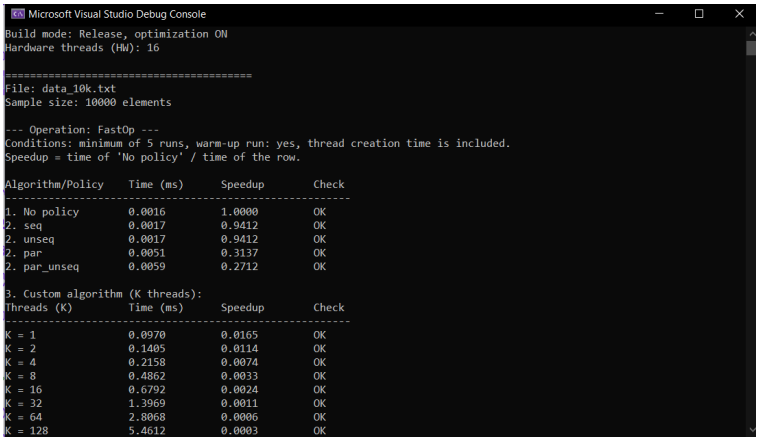
Власний алгоритм проти бібліотечного. par і par_unseq не гірші за наш алгоритм: на 10k швидші на ≈36 %, на 1M однакові, на 10M швидші на 18–19 %. Ймовірно, бібліотека використовує пул потоків і розподіляє роботу рівномірніше.
Залежність часу від K і закон зростання До K = 16 (кількість логічних потоків) час зменшується майже обернено пропорційно K: для SlowOp, 10M, /O2 при K = 2, 4, 8, 16 прискорення дорівнює 2,0, 4,0, 7,1 і 9,4 раза. Після K = 16 виграшу немає, а час росте лінійно: $T(K) \approx a + c \cdot K$, де $c \approx 0{,}045$ мс — вартість створення одного потоку (для 10 000 елементів при K = 512 час 24,5 мс в обох режимах). На великих даних (SlowOp, 10M) цей ріст малопомітний: від K = 16 до K = 512 час збільшується лише на 4–6 %.

Вплив оптимізації FastOp при /O2 швидша, ніж при /Od, у 1,4–11 раза: приблизно в 10–11 разів на 10k і 100k та в 1,4–1,7 раза на 1M і 10M (там, ймовірно, швидкість обмежує пам'ять). SlowOp виграє лише ≈11 % на всіх розмірах, бо майже весь час іде на бібліотечні sin і cos. Оптимізація змінює й користь від потоків: для FastOp на 10M при /Od par дає 1,6–2,1 раза, а при /O2 — лише 1 раз.

Додатки:



(білд солюшена)



(скріншот вікна консолі)